

Take-Home Assignment: GenAI Automation Workflow Engineer

Time budget: 3 days (we don't track hours, but we expect this to take ~15–20 hours of focused work) **Submission:** GitHub repo + 5–8 min Loom/video walkthrough + deployed demo (optional)

1. Why this assignment exists

We're hiring engineers who can ship **GenAI workflows that actually work in production**, not demos that pass a happy-path test and break the moment a real user touches them.

This assignment is intentionally open-ended. We are evaluating *how you think* at least as much as *what you build*. There is no single right answer. The strongest submissions will demonstrate clear engineering judgment, taste in UX, and an honest understanding of what could go wrong in the real world.

If you find yourself building something obvious and bloated, stop and reconsider. We'd rather see a smaller, sharper system than a feature-complete mess.

2. The problem

You are building a tool for a fictional mid-sized SaaS company, **Lumen**, that receives ~500 inbound customer support emails per day. Their support team is drowning. They want a GenAI-powered workflow that:

1. **Ingests** an inbound support email (subject + body + sender metadata).
2. **Classifies** it across at least three useful dimensions (e.g., category, urgency, sentiment, customer tier, you decide what's actually useful).
3. **Enriches** it by retrieving relevant context: past tickets from the same customer, relevant help-center articles, account status. **Use real integrations with free-tier tools for these data sources, not in-memory dictionaries.** Notion is an obvious fit for a help-center / knowledge base. Airtable or Google Sheets work for customer accounts and ticket history. Supabase or a local Postgres also work. Pick whatever you can stand up in an hour. The *integration* must be real, with real API auth, real network calls, real rate limits and failures your workflow has to handle gracefully. Populate the sources with synthetic data. Nothing in this assignment should cost you money to set up. Include a seed script and setup instructions so a reviewer can reproduce your environment.
4. **Decides** an action: auto-respond with a draft, route to a specific team, escalate to a human, or close as spam/no-action. The decision policy is yours to design and justify.
5. **Drafts** a response when appropriate, grounded in the retrieved context.
6. **Surfaces** all of this to a human operator through a dashboard where they can review, edit, approve, or override the agent's decision before anything goes out.

You will need to **generate or synthesize** a realistic dataset of at least 50 inbound emails covering a wide spread of cases, including adversarial and edge cases (angry customers, ambiguous requests, multi-issue tickets, prompt injection attempts, non-English snippets, etc.). The quality of your test data is part of what we evaluate.

3. What we want you to build

3.1 The workflow (backend)

A real, runnable pipeline, not a notebook. We expect:

- A clean separation between orchestration logic, prompts, tools/retrieval, and side effects.
- Prompts version-controlled in the repo, not hidden in string literals scattered across the codebase.
- Structured outputs from the LLM (function calling, JSON mode, or equivalent), not regex-parsing freeform text.
- Retries, timeouts, and graceful degradation when the LLM fails or returns garbage.
- Logging/tracing of every step of every run, in a form you can later query and replay.
- A clear answer to: "what does this workflow do that deterministic code cannot?" That is, you have not used an LLM where a regex or a database lookup would have done the job better.

Use self-hosted, lightweight open-source models. Avoid hosted LLM APIs (OpenAI, Anthropic, Gemini, etc.) as the backbone of your workflow. Reach for Ollama, llama.cpp, vLLM, HuggingFace transformers, or whichever local runtime you prefer. Pick the smallest model that does the job: a 3B-8B chat model for drafting, a dedicated classifier or embedding model for routing, a sentence-embedding model for retrieval, a small reranker. Use a larger model only when you can show a smaller one fails.

This constraint is deliberate. Anyone can wire up an API call. We want to see whether you can make small models punch above their weight through good prompting, retrieval, decomposition, and routing, and whether you can pick the right model for each sub-task instead of throwing one general-purpose model at every problem.

You are also encouraged to reach beyond pure text where it genuinely adds value, for example Whisper (or whisper.cpp / faster-whisper) for inbound voice notes attached to tickets, a small vision model for screenshot attachments, or a local TTS for an accessibility mode on the dashboard. Do this only if it makes the product meaningfully better, not for show.

If a hosted API is genuinely necessary for one specific sub-step and you can justify it in writing, that is acceptable, but the core decisioning and drafting must run locally on commodity hardware. We should be able to clone your repo and run the system without

paying anyone for tokens.

You may use any orchestration framework or none. We care that you can defend each choice.

3.2 The dashboard (frontend)

This is where we separate engineers who build for themselves from engineers who build for users. The dashboard is for a **support team lead**, someone non-technical, busy, and skeptical of AI. They will never open a terminal. They will never read your README. They should not have to.

The single thing we are evaluating here is: **how much engineering load have you taken off the end user?** Every config file you make them edit, every CLI command you make them run, every error message that says "check the logs", every silent failure that requires a developer to debug, is a point against you. Anything that a real support lead would need to ping an engineer about is a failure of design.

Minimum:

- A working **login and logout flow** with at least two roles (e.g., support agent and team lead) and visibly different permissions between them. Auth does not have to be production-grade, but it must actually work, persist sessions sensibly, and lock things down for unauthenticated users. Hardcoded users in a config file are fine; a real `/login` page that is ugly is better than a polished page that does nothing.
- A queue view of incoming tickets and the agent's proposed action for each.
- A detail view that shows, for any ticket, the full reasoning trace: what was retrieved, what the model classified it as, what it drafted, and why it chose its action.
- The ability to approve, edit, or reject the proposed response/action.
- Some form of aggregate view: how is the agent performing over time, where is it failing, what's the human override rate.
- Sensible empty states, loading states, and error states. When something breaks, the user should know what to do next without calling an engineer.

Vibe-coding the UI is allowed and expected. Use whatever AI tooling gets you to a clean result fast. We do not want you spending half your time hand-rolling CSS. What we do want is evidence that you made deliberate choices about *what* the user sees, *when*, and *why*, even if the visual layer was generated. If your dashboard looks like every other AI-generated dashboard, that is a signal you did not push back on the defaults.

The dashboard does not need to be beautiful. It needs to be **legible, fast, trustworthy, and operable by someone who has never seen it before**. A team lead should be able to log in, glance at it, and know whether to trust the agent today, all without asking a question.

We will judge your UX choices harder than your code here. A generic AI-generated dashboard with no information hierarchy is not enough. Show us you thought about the human on the other end.

3.3 Evaluation harness

This is non-negotiable. Build an offline evaluation system that:

- Runs your workflow over a held-out test set.
- Reports metrics that *you* have decided matter for this product, with a written justification of why those metrics matter and what their limits are.
- Includes at least one form of qualitative evaluation (LLM-as-judge, rubric grading, side-by-side comparison, your call) alongside any quantitative metrics.
- Is reproducible: a single command should regenerate the evaluation report.

If you ship a workflow without an eval harness, we will assume you do not yet know how to build production GenAI systems. This is the single highest-signal piece of the assignment.

3.4 GenAI fundamentals we expect you to engage with

This assignment is also a test of whether you understand the building blocks of modern GenAI systems, not just how to call an LLM API. We expect your design and your write-up to engage thoughtfully with each of the following. You do not have to *use* every one of these (sometimes the right answer is "I chose not to, here's why"), but you should *demonstrate* that you understand them and made deliberate choices.

- **Retrieval (RAG).** How you chunk, embed, store, retrieve, and rank the supporting context (past tickets, help-center articles, account data). Justify your choice of embedding model, chunking strategy, top-k, and whether you re-rank. Show that you can evaluate retrieval quality separately from end-to-end quality. Cite sources in your drafts so a human can verify them.
- **Memory.** Distinguish between short-term memory (within a single ticket thread) and long-term memory (across a customer's history). Decide what is worth remembering, what gets summarized, and what gets discarded. If your system handles a customer who replies to the drafted response, show how prior context is carried forward without dragging the entire history into every prompt.
- **Context engineering.** What goes into the prompt, in what order, and what gets cut when the context grows. Demonstrate awareness of context window limits, position bias (lost-in-the-middle), and the cost of stuffing irrelevant tokens.
- **Prompts.** Versioned, tested, and reasoned about. Show at least one prompt you iterated on, with notes on what changed and why. System vs user vs few-shot examples should be deliberate choices, not accidents. No magic incantations.
- **Structured output.** Use function calling, JSON schemas, or constrained generation wherever downstream code needs structure. Validate outputs, do not trust the model to always comply.
- **Tool use and the reason-act loop.** Your decisioning step is naturally agentic. If you implement it as a ReAct-style loop with tools (`retrieve_history`, `lookup_account`, `classify`, `draft_response`, `escalate`), explain the trade-offs against a simpler one-shot pipeline. Either choice is defensible, an unexamined choice is not. If you do build a

loop, cap iterations and handle the model getting stuck.

- **Guardrails.** Both input-side and output-side. Email bodies are user-controlled and can contain prompt injection. Drafts can leak PII, make promises the company cannot keep, hallucinate policy, or respond in inappropriate tone. Demonstrate concrete defenses, not just a paragraph saying "we should sanitize inputs."
- **Model routing and right-sizing.** It is almost never correct to send every request to the same model. Show judgment about which local model handles which sub-task: a tiny classifier for triage, an embedding model for retrieval, a small chat model for drafting, a stronger one only for genuinely hard cases. A multi-model pipeline that uses each model for what it is good at beats one general-purpose model on every axis that matters, latency, memory, accuracy, cost.
- **Grounding and citations.** Anything the model claims about the customer's account, past tickets, or policy should be traceable back to a retrieved source. The dashboard should make this visible to the reviewer.
- **Observability.** Per-run traces with token counts, latencies, retrieved chunks, tool calls, and prompt versions should be inspectable. The eval harness depends on this. The dashboard's debug view depends on this.
- **Failure handling.** Timeouts, retries with backoff, schema-validation failures, model refusals, partial tool failures, rate-limits. The system should degrade visibly, not silently.

Treat this section as the **substance of what we are hiring for**. A submission that ships a working dashboard but cannot articulate its choices across these dimensions is weaker than one that ships less but reasons about each carefully.

4. Things we are explicitly looking for

In rough priority order:

1. **Clarity of thinking about success.** Before any code, your README should answer: what does "this works" mean? What metrics? What thresholds? What's the cost/latency budget per ticket and why? What does a failure look like?
2. **Failure-mode awareness.** Document the top 5 ways this system can go wrong in production and what you've done (or would do) about each. Hallucinated facts, prompt injection from email bodies, PII leakage, cost runaway, biased classification, and silent degradation are all fair game.
3. **Engineering depth.** Sensible abstractions, testable units, dependency injection where it matters, no god-objects, no copy-pasted prompts. Code we'd be comfortable inheriting.
4. **Honest scope.** A small, polished, well-evaluated workflow beats a sprawling one with no evals. Cut features ruthlessly. Tell us what you cut and why.
5. **UX judgment on the dashboard.** Information density, trust signals (showing confidence, sources, edit history), sensible empty/loading/error states, and a clear path for the human to take over. We want to see that you've absorbed the engineering

for the human to take over. We want to see that you've absorbed the engineering complexity yourself instead of leaking it onto the user. Bonus for thinking about how a tired operator at 4pm interacts with this.

6. **Prompt discipline.** Prompts that are versioned, testable, and reasoned about, not magic incantations. We want to see at least one prompt you iterated on, with notes on what changed and why.
 7. **Cost and latency awareness.** Tell us the per-ticket cost and p95 latency of your workflow. Tell us how you'd cut each by half.
 8. **Documentation taste.** A README that respects our time. No 800-word marketing intro. Get to the point.
-

5. Things we don't care about

- Whether you used LangChain, LlamaIndex, raw SDKs, or wrote your own thin wrapper.
- Whether you used a vector DB, SQLite with FTS, or grep. Use the simplest thing that works and explain why.
- Whether your dashboard is React, Streamlit, Gradio, htmx, or server-rendered HTML.
- Whether your code is in Python, TypeScript, Go, or Rust.
- Test coverage percentage. We care that the *important* paths are tested, not that you hit 90%.
- Cloud deployment. Localhost is fine. A deployed demo is a small bonus, not a requirement.

If you spend time gold-plating any of the above instead of the eval harness or the failure-mode analysis, you've optimized for the wrong thing.

6. Stretch goals (optional, only if the core is solid)

Pick at most one or two. We'd rather see depth than breadth.

- A self-improvement loop: capture human edits/overrides and use them to improve prompts or routing rules.
- A regression-detection system that flags when a prompt change degrades performance on the eval set.
- Adversarial robustness: a small red-team suite of malicious inputs (prompt injection, jailbreaks, data exfiltration attempts) and how your system handles each.
- A cost dashboard that breaks down spend per workflow stage and per ticket category.
- Fine-tuning, LoRA-adapting, or distilling a small local model on synthesized training data so it outperforms a larger off-the-shelf model on a specific sub-task (e.g., classification or routing). Show the before/after numbers.

7. Deliverables

Submit all of the following:

1. **A public GitHub repository** containing:
 - All code (backend, frontend, eval harness, data generation scripts).
 - A `README.md` that includes: setup instructions, your success metrics and justification, failure-mode analysis, cost/latency numbers, a list of trade-offs you made and why, and what you'd build next with another week.
 - A `PROMPTS/` directory (or equivalent) with versioned prompts.
 - An `EVALS/` directory with the eval harness, the test set, and a sample report.
 - A `.env.example` so we can run it.
2. **A 5-8 minute video walkthrough** (Loom, YouTube unlisted, or attached file) covering:
 - A live demo of the workflow on 2-3 sample tickets, including at least one where it fails or struggles.
 - A walk through the dashboard from the perspective of the support lead.
 - The eval harness running and what you learned from its output.
 - The single design decision you are most proud of, and the single one you are least sure about.

Do not script this. We'd rather hear you think out loud than watch a polished sales pitch.
3. **A short written reflection** (`REFLECTION.md`, ~500 words) covering:
 - What you would do differently with another week.
 - What part of the system you trust the least and why.
 - How you would know, three months after launch, whether this system was a success or a quiet failure.
4. **(Optional)** A deployed demo URL.

8. Evaluation rubric

We score each submission across these dimensions on a 1-5 scale:

Dimension	What we look for
Problem framing	Did you understand and re-scope the problem thoughtfully, or did you just implement the spec literally?
Engineering quality	Code we'd want to maintain. Sensible architecture. No premature complexity.

Workflow design	Right use of LLM vs. deterministic logic. Structured outputs. Robust to failure.
GenAI fundamentals	Deliberate, justified choices across RAG, memory, context, prompts, tool use, guardrails, routing, and grounding. Depth of understanding, not feature checkboxing.
Eval harness	Exists, is reproducible, measures things that actually matter.
Failure-mode reasoning	Honest, specific, and connected to the implementation.
Dashboard UX	A non-technical user can log in and operate it without help. Trust signals, sensible states, no engineering load leaking through.
Cost & latency thinking	Numbers exist; trade-offs are articulated.
Communication	README, video, and reflection are clear and respect our time.

A score of 4+ in *GenAI fundamentals*, *failure-mode reasoning*, and *eval harness* is roughly the bar for moving forward, regardless of how shiny the rest is.

9. Ground rules

- You may use any AI coding assistant (Claude Code, Cursor, Copilot, etc.). We do this too. Be ready to defend every line in the interview, if you can't explain it, it doesn't count as yours.
- Do not exfiltrate or use real customer data from anywhere. Synthetic data only.
- If you have a question about scope, make a reasonable assumption, document it in the README, and move on. Do not email us asking for clarification, handling ambiguity is part of the test.
- Submit even if you didn't finish everything. A partial, honest submission with a clear write-up of what you cut and why is more valuable to us than silence.

10. How to submit

Email a link to your GitHub repo and your video to hoomanhr@hooman.digital with the subject line `GenAI Workflow Assignment: <Your Name>`.

We aim to respond within 5 business days. If you don't hear back in 7, please nudge us.

Good luck. Build something you'd be proud to ship.